



**Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»**

РТУ МИРЭА

Институт информационных технологий

Кафедра инструментального и прикладного программного обеспечения

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**Разработка веб-приложения на основе интеграционного паттерна Service Mesh для развития
информационного ландшафта крупных предприятий**

Студент: Верт Д.А.

Группа: ИКБО-24-20

Руководитель: Беляев П.В., доцент, к.т.н.

Консультант по экономической части: Чижанькова И.В., доцент, к.э.н.

Москва 2024



Цель и задачи

Цель:

1. Разработка веб-приложения на основе интеграционного паттерна Service Mesh.
2. Анализ проблематики использования классических методов межсервисной интеграции в сравнении с преимуществами перехода на Service Mesh.

Объект исследования:

1. Подходы обеспечения межсервисной интеграции.

Предмет исследования:

1. Устройство и преимущества интеграционного шаблона Service Mesh.

Актуальность:

1. Перспективный путь развития и масштабирования межсервисных интеграций внутри ИТ-компаний.

Новизна:

1. Сравнение инновационного способа интеграции сервисов с традиционными подходами.
2. Веб-приложение позволит бесшовно внедриться на пути выполнения сетевых запросов между двумя сервисами и управлять доступностью одного из них внутри кластера в режиме реального времени.



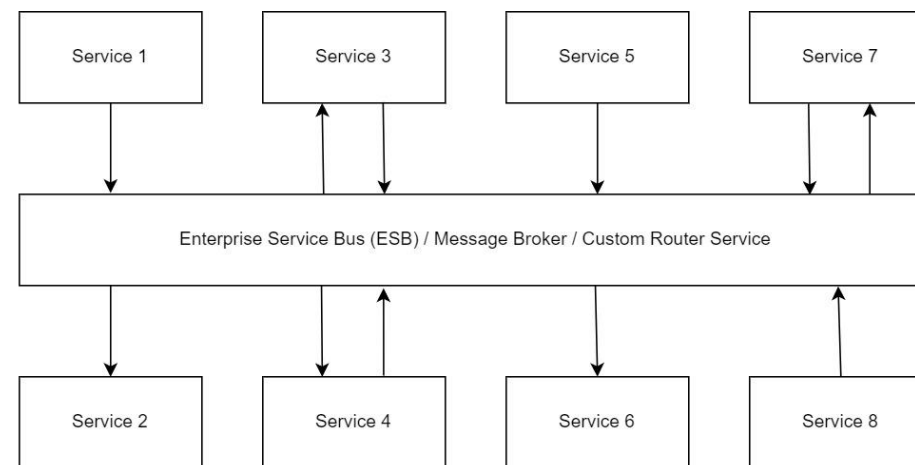
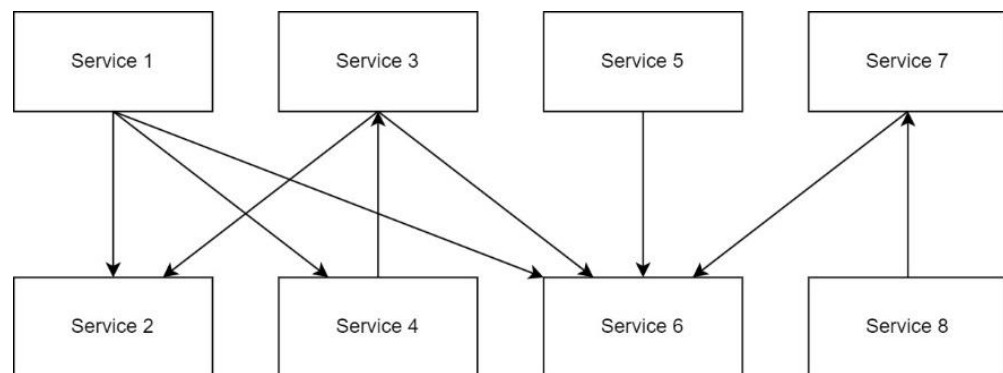
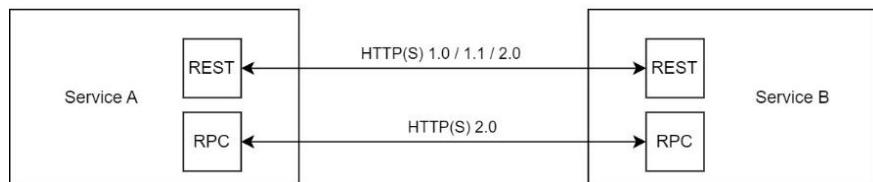
Гипотеза:

1. Service Mesh в определенных условиях решает недостатки традиционных подходов интеграции сервисов и привносит дополнительные возможности управления, контроля и мониторинга распределенных систем.

Задачи:

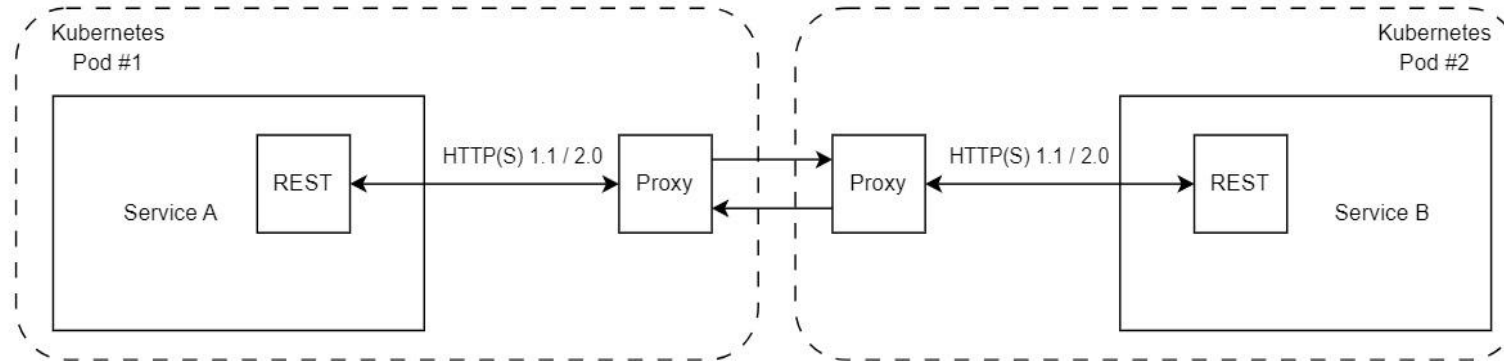
1. Исследование существующих технологических решений для выполнения межсервисной интеграции.
2. Анализ проблем и ограничений применения традиционных подходов.
3. Обзор интеграционного паттерна Service Mesh и реализующих его фреймворков.
4. Постановка задачи на разработку распределенного веб-приложения на основе данного шаблона.
5. Обоснование и выбор инструментов разработки.
6. Проектирование адаптированной модели жизненного цикла.
7. Проектирование веб-приложения на основе интеграционного паттерна Service Mesh.
8. Разработка веб-приложения.
9. Сборка и развертывание всех составных микросервисов приложения в кластере.
10. Установка Istio Service Mesh, настройка политик маршрутизации трафика.
11. Расчет вычислительной и емкостной сложности итогового веб-приложения.
12. Проведение тестирования разработанного программного продукта.
13. Расчет стоимости проведения работ.

Анализ предметной области

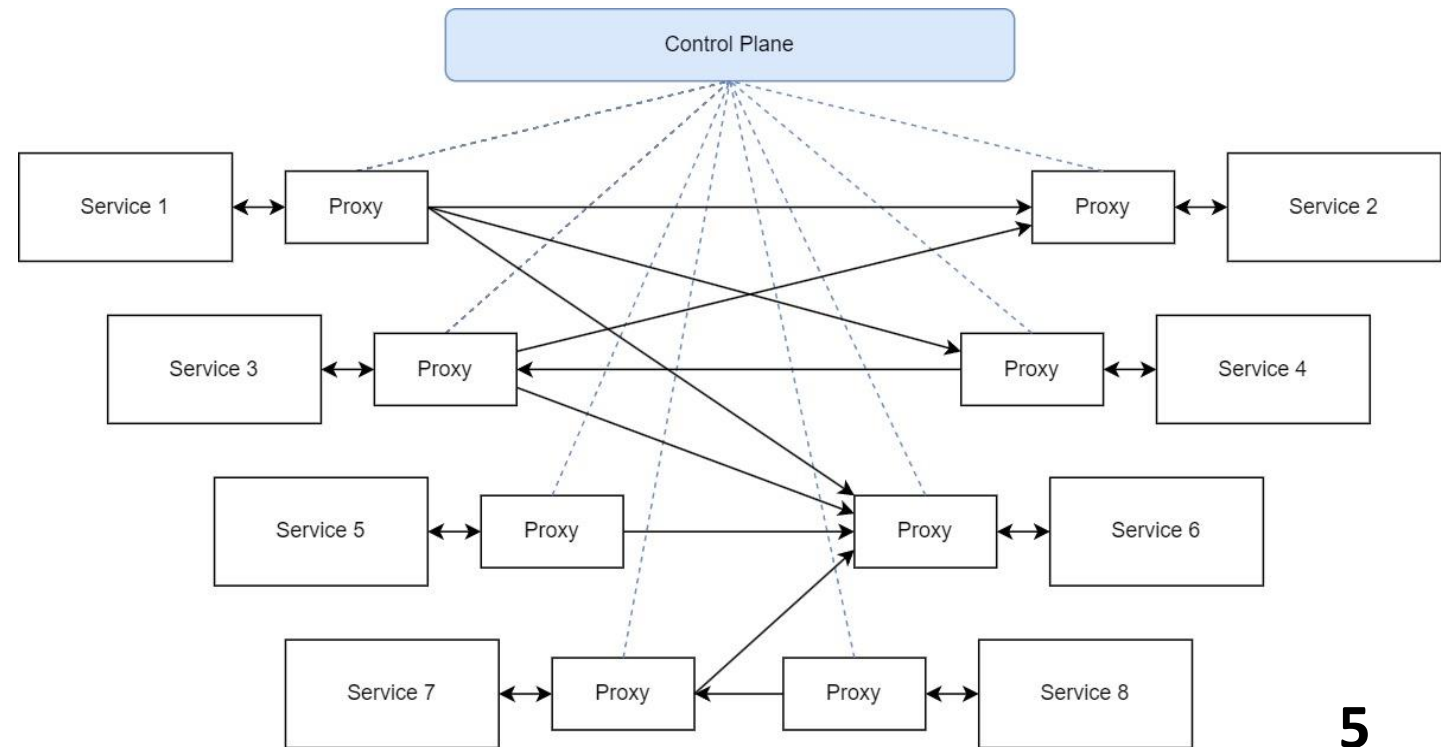


1. Сложность масштабирования.
2. Сильная связанность.
3. Отсутствие централизованного управления.

1. Риск единой точки отказа.
2. Сложность горизонтальной масштабируемости.
3. Проприетарная технология, зависимость от вендора.
4. Отсутствие мониторинга за состоянием всей системы.
5. Отсутствие возможности для добавления дополнительной логики.



1. Децентрализованность обработки запросов, но при этом единая точка конфигурации.
2. Архитектурно заложенное горизонтальное масштабирование.
3. Встроенные инструменты мониторинга и безопасности.
4. Автономность – Service Mesh разворачивается в существующем кластере и никак не внедряется в код приложений.
5. Open Source – никакой зависимости от иностранного вендора ПО.





Сравнительная таблица наиболее распространенных фреймворков для Service Mesh

Критерий	Istio	Linkerd	Consul
Управление трафиком	9	7	7
Безопасность	9	7	8
Наблюдаемость	9	8	6
Архитектура	8	8	7
Экосистема и сообщество	10	7	8
Легковесность и простота	7	9	7
Обнаружение сервисов	8	7	9
Поддержка облачных платформ	7	7	9
Сумма	67	60	61



Постановка задачи на разработку



- **Микросервис погоды (weather)** – должен взаимодействовать с запросами пользователя, с последующим обращением за этой информацией к сервису справочных данных.
- **Микросервис справочных данных (dictionary)** – должен хранить в своей базе данных сведения о погоде и предоставлять их с применением определенной выборки.
- **Микросервис управления трафиком (circuit-breaker)** – должен перехватывать запрос от сервиса погоды и либо пропускать его дальше до сервиса справочных данных, либо сразу прерывает их сетевое взаимодействие с заготовленным системным ответом.
- **Микросервис авторизации (auth)** – должен обеспечивать выдачу JWT токенов и авторизует по ним как пользователей веб-приложения, так и другие микросервисы между собой.
- **Веб-приложение** должно предоставлять пользовательский интерфейс для обычного пользователя и администратора.

Выбор инструментов разработки



PostgreSQL



kubernetes



Критерий	PostgreSQL	MySQL	MS SQL Server	SQLite	Критерий	Redis	Mongo DB	Couch base	Cassandra	Критерий	Kubernetes	Docker Swarm	Apache Mesos	Amazon ECS
Производительность	9	7	9	6	Производительность	9	8	7	8	Масштабируемость	9	7	7	8
Масштабируемость	9	8	9	7	Масштабируемость	8	7	9	10	Гибкость настройки	8	7	9	7
Гибкость модели данных	9	8	8	7	Гибкость модели данных	8	9	8	7	Управление ресурсами	8	9	8	8
Поддержка транзакций	9	8	9	7	Поддержка транзакций	5	7	7	9	Управление хранилищем	8	7	7	9
Сложность установки и настройки	8	8	7	10	Сложность установки и настройки	9	8	7	6	Мониторинг и логирование	9	8	7	7
Запросы и индексация	9	8	9	7	Запросы и индексация	9	8	7	7	Сообщество и поддержка	10	8	7	7
Сообщество и поддержка	9	9	8	7	Сообщество и поддержка	9	9	7	7	Общая оценка	52	46	45	45
Общая оценка	62	56	59	51	Общая оценка	57	55	52	54					

Проектирование веб-приложения



Адаптированная модель жизненного цикла



Диаграмма прецедентов

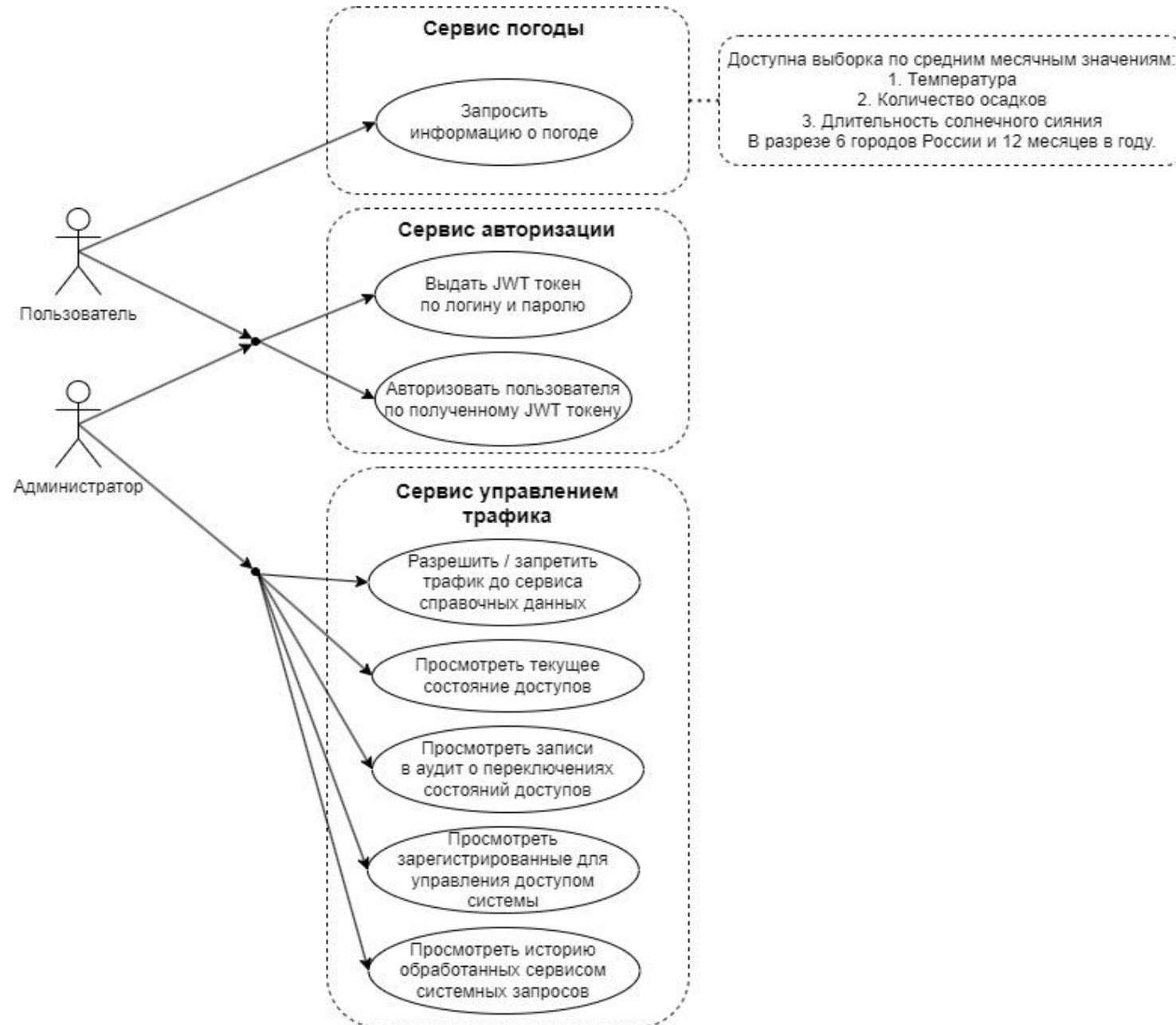
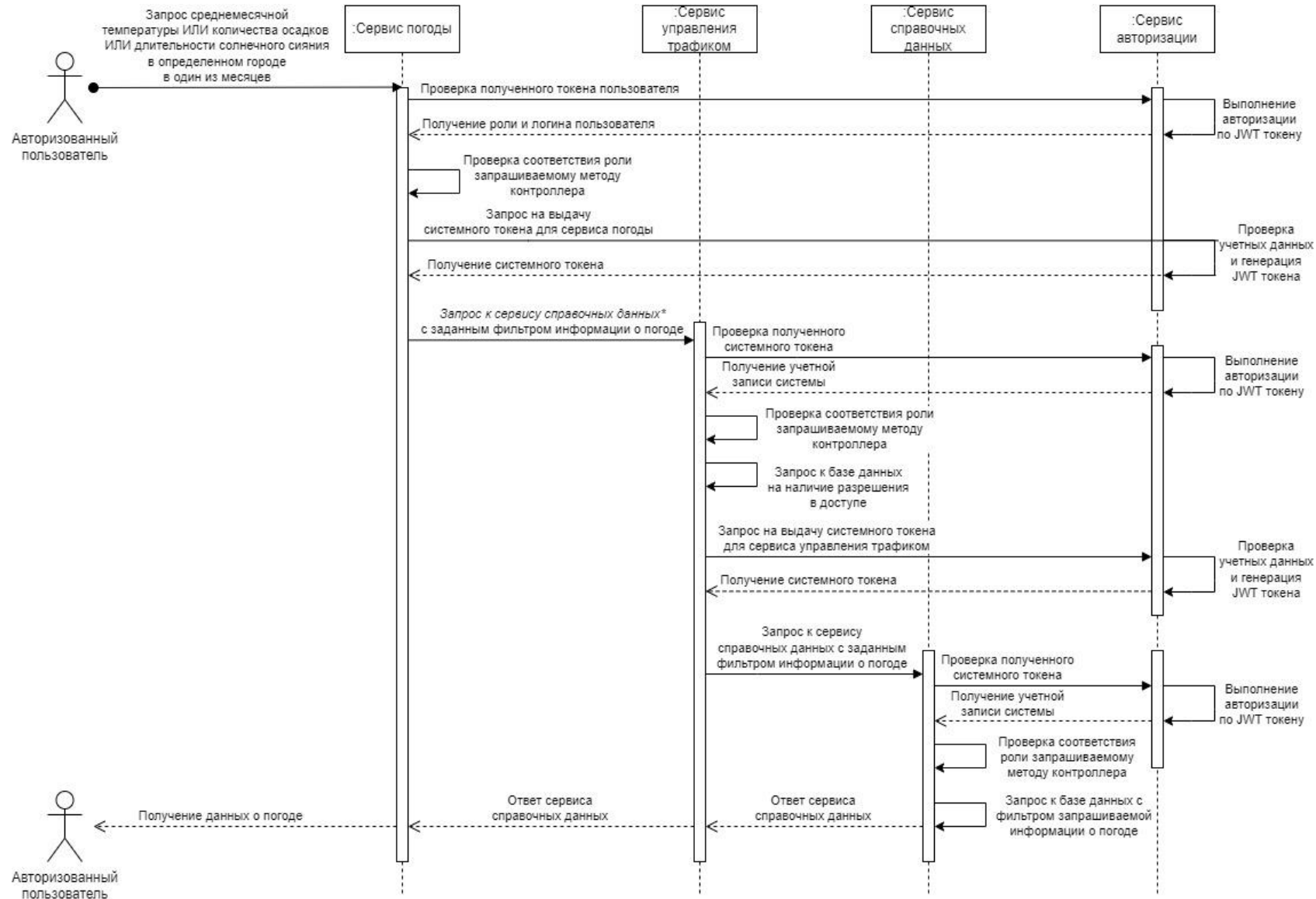


Диаграмма последовательности



*Запрос со стороны сервиса погоды направляется именно к сервису справочных данных. Однако настроенная политика маршрутизации Istio неявно перенаправляет этот запрос сервису управления трафиком.

Схема архитектуры веб-приложения

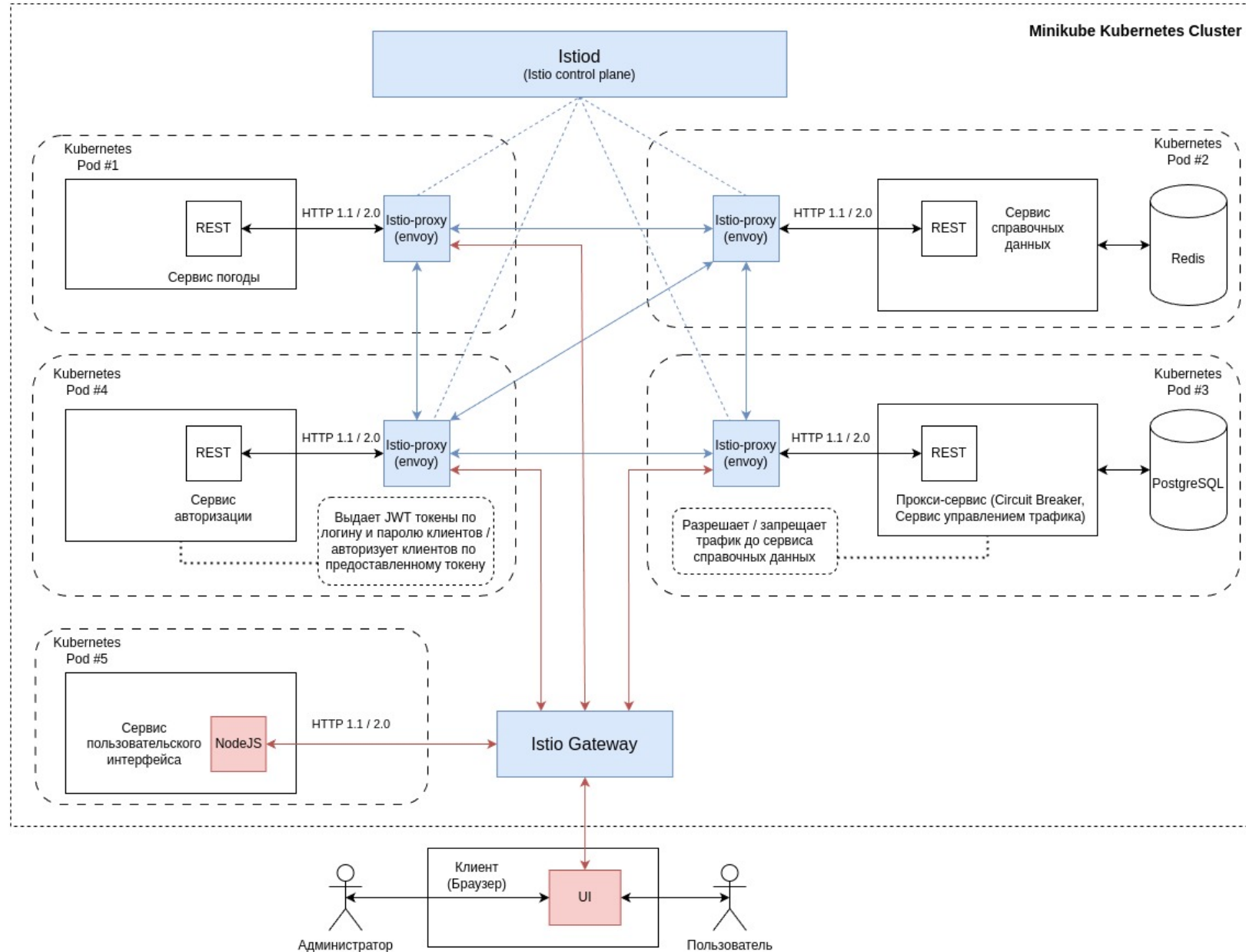
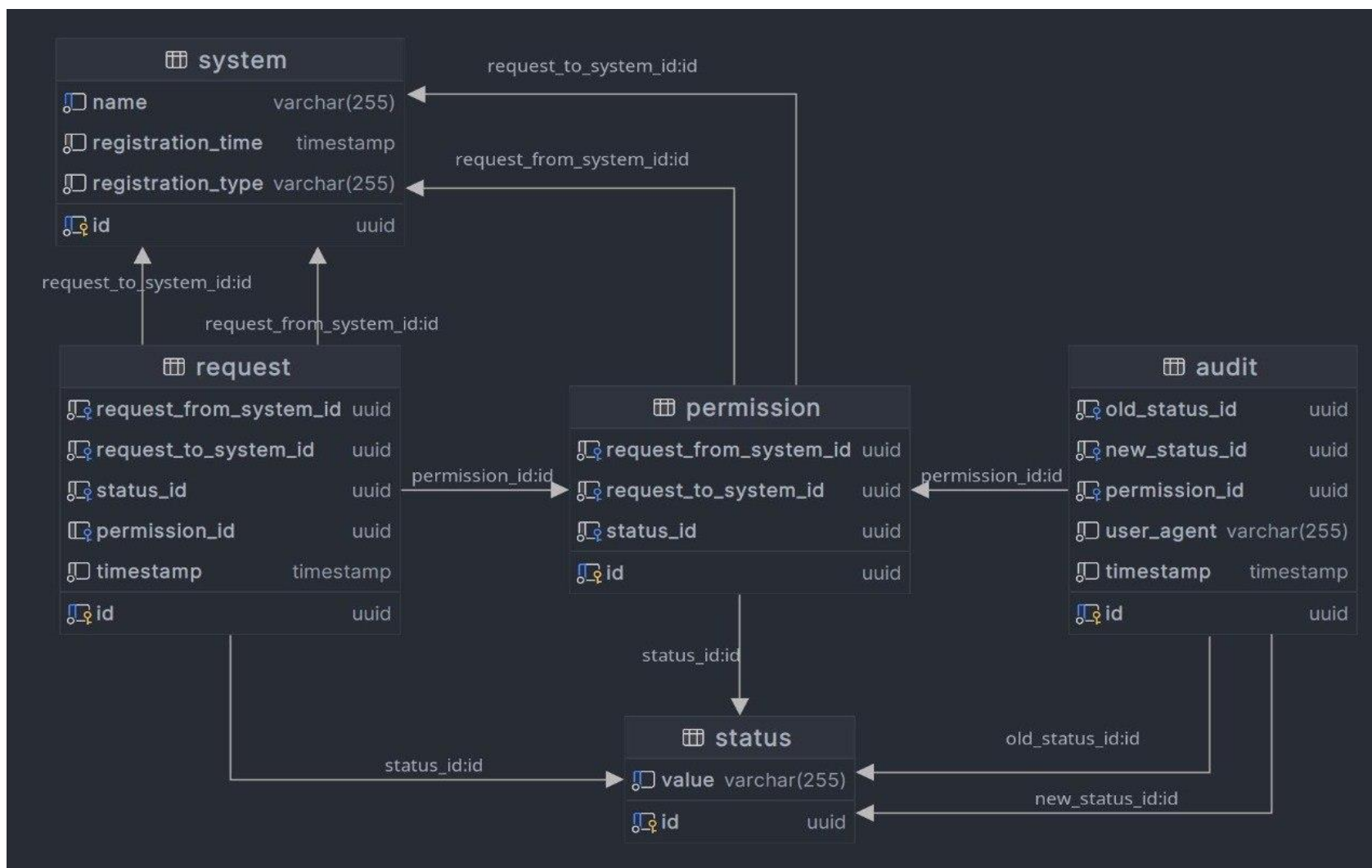




Диаграмма ER-D



Разработка веб-приложения



```
@RequestMapping("/weather")
public class WeatherController {

    @Value("${hostname.dictionary}")
    private String dictionaryServiceHost;

    @PostMapping("/info")
    public ResponseEntity<DictionaryResponseDto> getWeatherInfo(...) {
        ...

        var dictionaryServiceResponse = RestClient.create().post()
            .uri(dictionaryServiceHost + "/dictionary/info")
            .header(HttpHeaders.AUTHORIZATION, token)
            .header("request-from", "weather")
            .header("request-to", "dictionary")
            ...

        return ResponseEntity.ok(dictionaryServiceResponse);
    }
}
```

Микросервис погоды (weather)

URL: <http://gateway-ip/weather/info>

```
@RequestMapping("/circuit-breaker/processing/dictionary")
public class DictionaryProcessorController {

    @Value("${hostname.dictionary}")
    private String dictionaryServiceHost;

    @PostMapping("/info")
    public ResponseEntity<DictionaryResponseDto> processInfoRequest(...) {
        ...

        var processingResult = circuitBreakerService.tryProcessRequest(request);

        if (!processingResult.getIsAllowed()) {
            return ResponseEntity.ok(generateCircuitBreakerResponseDto());
        }

        var dictionaryServiceResponse = RestClient.create().post()
            .uri(dictionaryServiceHost + "/dictionary/info")
            .header(HttpHeaders.AUTHORIZATION, token)
            .header("request-from", "circuit-breaker")
            .header("request-to", "dictionary")
            ...

        return ResponseEntity.ok(dictionaryServiceResponse);
    }
}
```

Микросервис управления трафиком (circuit-breaker)

URL: <http://gateway-ip/circuit-breaker/processing/dictionary/info>



Маршрутизация трафика в Istio

```
1 apiVersion: networking.istio.io/v1alpha3
2 kind: VirtualService
3 metadata:
4   name: dictionary-virtual-svc
5 spec:
6   hosts:
7     - dictionary-svc
8   http:
9     - match:
10       - sourceLabels:
11           app: circuit-breaker-app
12       route:
13         - destination:
14             host: dictionary-svc
15     - match:
16       - uri:
17           prefix: /dictionary/info
18       rewrite:
19         uri: /circuit-breaker/processing/dictionary/info
20       route:
21         - destination:
22             host: circuit-breaker-svc
```

Манифест Virtual Service в Istio применяется для создания маршрутов сайдкара istio-проху.

Здесь он запрещает всем, кроме микросервиса управления трафиком (app: circuit-breaker-app) отправлять запросы к микросервису справочных данных (dictionary-svc) и **принудительно маршрутизирует** каждый такой запрос на проверку наличия открытого доступа на URI: /circuit-breaker/processing/dictionary/info



Расчет вычислительной и емкостной сложности

Микросервис	Метод API	Функционал	Вычислительная сложность (T)	Емкостная сложность (V)
Погоды (weather)	getWeatherInfo	Создание запроса к сервису справочных данных о погоде	$O(1)$	$O(1)$
Управления трафиком (circuit-breaker)	updatePermission	Обновление состояния доступа	$O(\log n)$	$O(n)$
	getPermission	Получение состояния доступа	$O(\log n)$	$O(1)$
	getAllSystems	Получение списка зарегистрированных систем	$O(n)$	$O(1)$
	getAllAuditEvents	Получение списка событий аудита	$O(n)$	$O(1)$
	getAllRequests	Получение списка обработанных запросов	$O(n)$	$O(1)$
	processInfoRequest	Обработка перехваченного запроса данных о погоде	$O(\log n)$	$O(n)$
Справочных данных (dictionary)	getDictionaryInfo	Поиск информации о погоде в сервисе справочных данных	$O(n)$	$O(1)$
Авторизации (auth)	createToken	Создание JWT-токена по авторизационным данным	$O(1)$	$O(1)$
	getUserByToken	Предоставление авторизационных данных по JWT-токену	$O(1)$	$O(1)$

Тестирование веб-приложения



Weather-Mesh-UI default_user USER

Авторизация
Выйти из системы

Погода
Поиск

Управление трафиком
Управление доступами
История запросов
Системы
Аудит

Дополнительно
О системе
Об авторе
Ссылки

Запрос данных о погоде

Условия погоды: Температура
Месяц: Май
Город: Москва

Поиск!

Weather-Mesh-UI default_user USER

Авторизация
Выйти из системы

Погода
Поиск

Управление трафиком
Управление доступами
История запросов
Системы
Аудит

Запрос данных о погоде

Условия погоды: Температура
Месяц: Май
Город: Москва

Поиск!

Значение: 13.6

Описание: Среднемесячная температура воздуха, °C



Weather-Mesh-UI

default_adminADMIN

Авторизация

Выйти из системы

Погода

Поиск

Управление трафиком

Управление доступами

История запросов

Системы

Аудит

Дополнительно

О системе

Об авторе

Ссылки

Редактирование доступа

ID	From	To	Status	Submit
0d365b71-b49b-43ed-aeaa-2286bd9e98a8	weather	dictionary	ALLOWED	☒

Weather-Mesh-UI

default_adminADMIN

Авторизация

Выйти из системы

Погода

Поиск

Управление трафиком

Управление доступами

История запросов

Системы

Аудит

События аудита

ID	From	To	Old status	New status	Username	Timestamp
9110d215-06c3-446b-a605-fbb293d0f376	weather	dictionary	ALLOWED	FORBIDDEN	default_admin	2024-05-14 19:26:51



Weather-Mesh-UI

default_userUSER

Авторизация

Выйти из системы

Погода

Поиск

Управление трафиком

Управление доступами

История запросов

Системы

Аудит

Дополнительно

О системе

Об авторе

Ссылки

Запрос данных о погоде

Условия погоды

Месяц

Город

Температура

Май

Санкт-Петербург

Поиск!

Значение:

Описание: Сервис справочных данных временно недоступен. Запрос обработан сервисом управления трафика в 2024-05-14 19:27:20

Weather-Mesh-UI

default_adminADMIN

Авторизация

Выйти из системы

Погода

Поиск

Управление трафиком

Управление доступами

История запросов

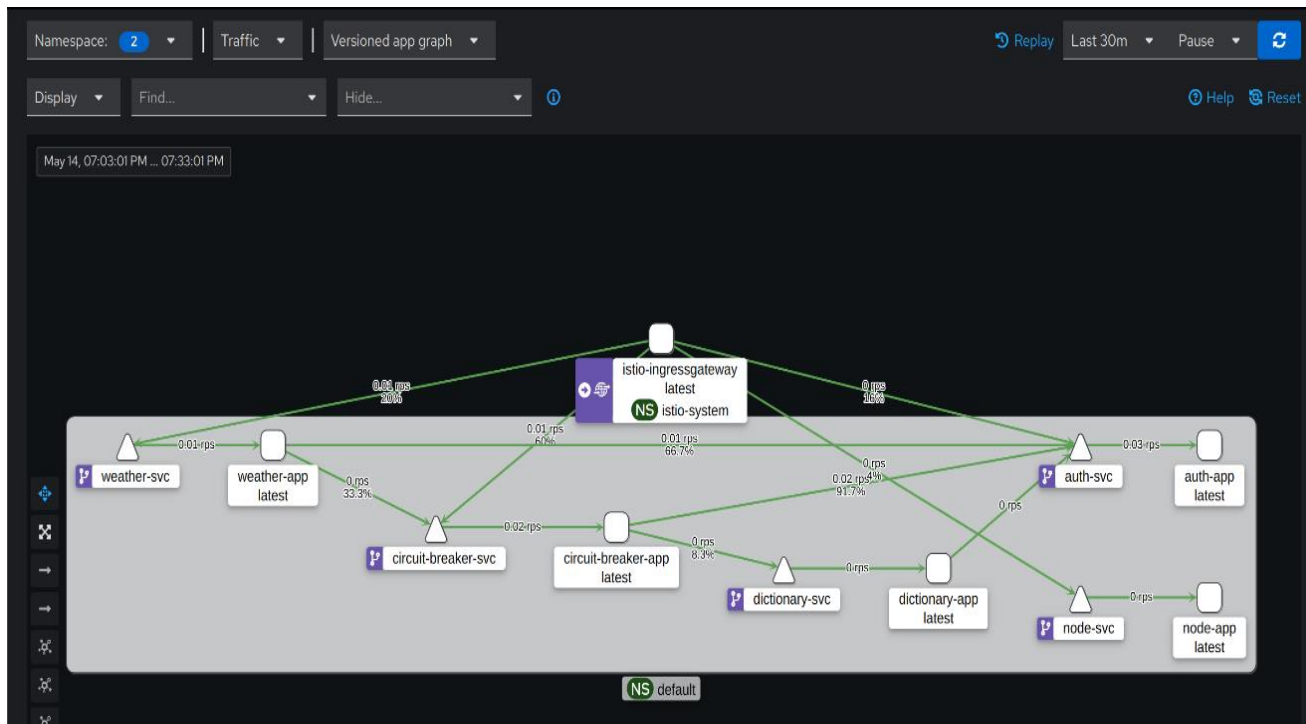
Системы

Аудит

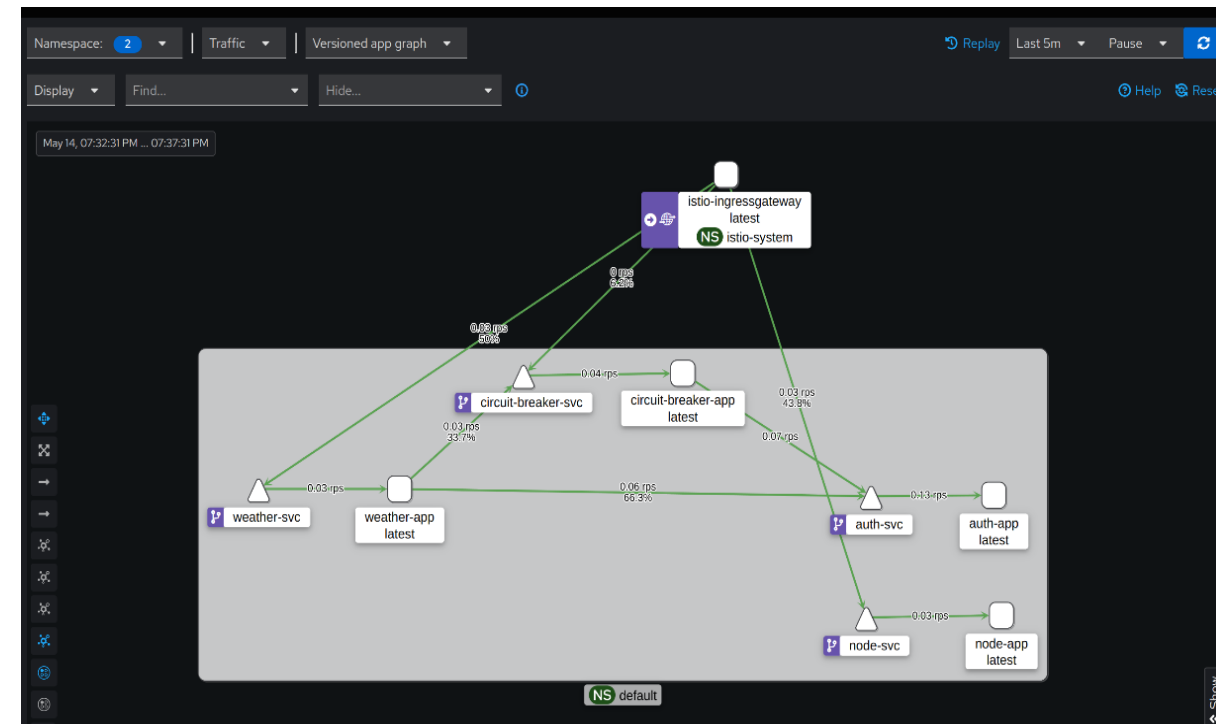
Обработанные запросы

ID	From	To	Status	Timestamp
d993b3fb-1b3b-488f-84a9-651d093d8091	weather	dictionary	ALLOWED	2024-05-14 19:25:43
78f83ec1-5518-4f49-a637-9c18de2973ee	weather	dictionary	FORBIDDEN	2024-05-14 19:27:20

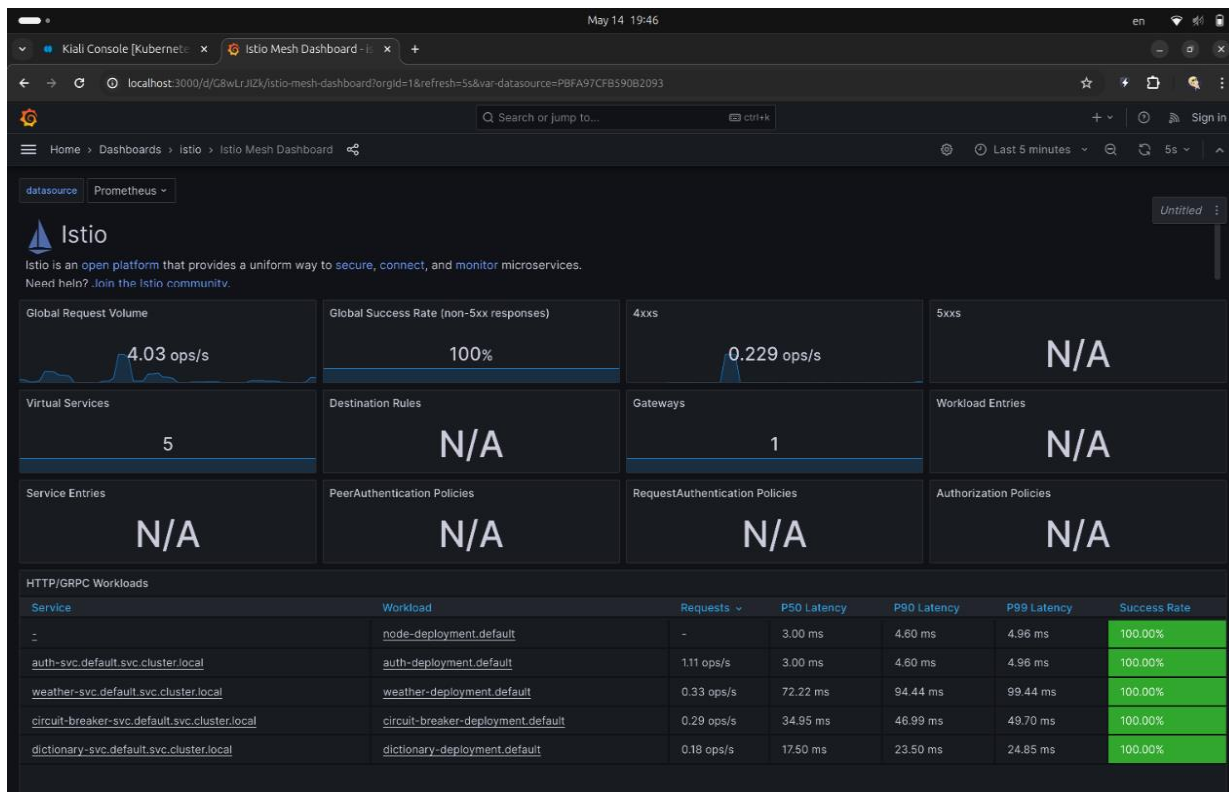
Мониторинг инструментами Istio



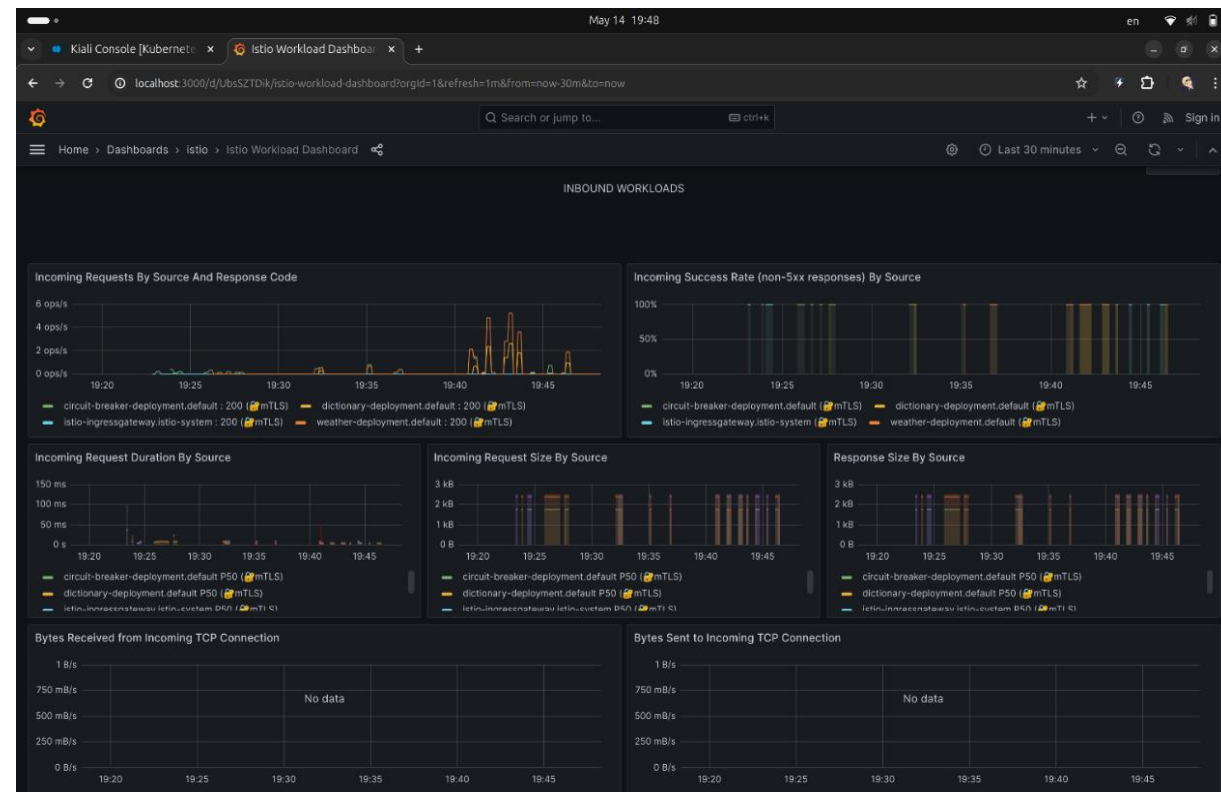
Визуализация течения трафика в Service Mesh при открытом доступе в сервисе управления трафиком



Визуализация течения трафика в Service Mesh при закрытом доступе в сервисе управления трафиком



Панель отображения сетевых метрик
в Grafana для Service Mesh

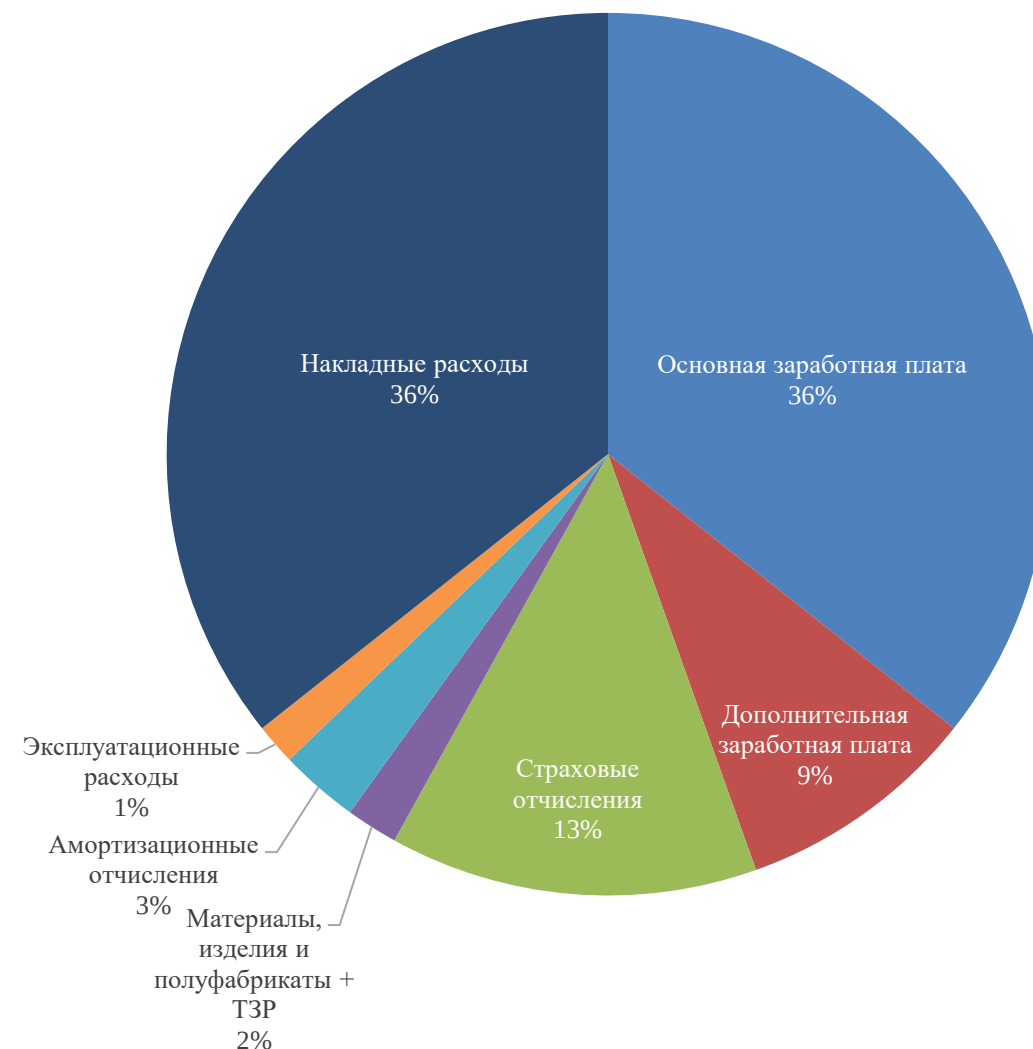


Панель отображения графиков
нагрузки на ресурсы в Grafana



Расчет экономической стоимости

№ п/п	Номенклатура статей расходов	Затрат ы (руб.)	Доля расход ов, %
1	Основная заработная плата	72800	35,7
2	Дополнительная заработная плата	18200	8,9
3	Страховые отчисления	27482	13,5
4	Материалы, покупные изделия и полуфабрикаты, а также до 20% от соответствующей суммы на транспортно-заготовительные расходы (ТЗР)	3850	1,9
5	Расходы на приобретение компьютерной техники и программного обеспечения, а также до 20% от соответствующей суммы на ТЗР	0	0
6	Амортизационные отчисления по используемому в проекте оборудованию	5790	2,8
7	Эксплуатационные расходы, связанные с использованием компьютерной техники	3088	1,5
8	Накладные расходы	72800	35,7
9	Прочие расходы	—	—
Итого:		204 010	100



Заключение

Все заявленные требования к веб-приложению были выполнены. Цели достигнуты и гипотеза подтвердилась, Service Mesh предлагает следующие решения недостаткам традиционных подходов:

1. **Сложность масштабирования** – в Service Mesh архитектурно заложено легкое горизонтальное масштабирование.
2. **Сильная связанность** – Service Mesh не требует никаких доработок в коде приложений и применяет свои настройки поверх текущего сетевого взаимодействия.
3. **Отсутствие централизованного управления** – Control Plane (Istiod в Istio), единый компонент в архитектуре Service Mesh, отвечает за хранение и применение конфигурации всей системы.
4. **Риск единой точки отказа** – Все запросы проходят через istio-proxy каждого из микросервисов, и если один из них выйдет из строя, то все остальные будут работать. Обрушить систему может только выход из строя Control Plane, что маловероятно.
5. **Проприетарная технология, зависимость от вендора** – фреймворк Istio Service Mesh полностью бесплатен и свободно распространяем.
6. **Отсутствие мониторинга за системой** – Istio поддерживает встроенные и гибко настраиваемые инструменты мониторинга и безопасности.
7. **Отсутствие возможности добавления дополнительной логики** – Istio позволяет максимально гибко управлять и настраивать сетевые политики, что достигается манифестами Istio в декларативном стиле.

Публикации



СПАСИБО ЗА ВНИМАНИЕ!